

MNIST Digit Classification Analysis

Model Training, Evaluation, and Visualization Workflow

1. Workflow Overview

- 1 Data Preparation:** Images are processed in their 2D (28x28) format.
- 2 Model Architecture:** Use of a *Flatten* layer to automatically vectorize input data.
- 3 Training:** Optimization via Adam and loss tracking with Sparse Categorical Crossentropy.
- 4 Evaluation:** Testing performance on held-out data.
- 5 Diagnostics:** Visualizing error patterns using a Confusion Matrix heatmap.

2. Implementation Details

Confusion Matrix Visualization

```
y_predicted_labels = [np.argmax(i) for i in y_predicted]
cm =
tf.math.confusion_matrix(labels=y_test, predictions=y_predicted_labels)

plt.figure(figsize = (10,7))
```

```
sn.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Truth')
```

How it works: `np.argmax(i)` finds the index of the highest probability in the prediction array (turning a vector of 10 values into a single digit label).
`tf.math.confusion_matrix` creates a grid comparing 'Truth' vs 'Predicted' to show where the model is getting confused.

Building the Optimized Model

```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dense(10, activation='sigmoid')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(X_train, y_train, epochs=10)
```

```
Epoch 1/10 - loss: 0.2959 - accuracy: 0.9185 ... Epoch 10/10 - loss: 0.0264 -
accuracy: 0.9917
```

Key Features:

- **Flatten Layer:** Eliminates manual `.reshape()`. It takes a (28, 28) matrix and flattens it to 784 pixels.
- **Dense (100):** A hidden layer with ReLU (Rectified Linear Unit) activation to learn non-linear patterns.
- **Dense (10):** Output layer with Sigmoid (or Softmax) to output probabilities for digits 0-9.
- **Sparse Categorical Crossentropy:** Chosen because `y_train` contains integers (0, 1, 2...) rather than one-hot encoded vectors.

Performance Evaluation

```
model.evaluate(X_test, y_test)
```

```
313/313 [=====] - loss: 0.0813 - accuracy: 0.9779
```

The final evaluation shows an accuracy of **~97.79%**. This indicates the model generalized well from the training data (99.17%) to the test data.